

CortexBrain

Guida strategica e tecnica per l'ottimizzazione data-driven dello scheduler Linux

Workload target: AI, HPC, Real-time | Ambiente: bare-metal edge | Tecnologie: eBPF, sched-ext, Rust, OpenTelemetry

1. Executive Summary

Il presente documento definisce la roadmap tecnica e strategica per evolvere CortexBrain da piattaforma di osservabilità eBPF-based in un **optimization engine** per il kernel Linux, focalizzato su workload AI, HPC e real-time eseguiti su infrastrutture bare-metal ed edge.

L'obiettivo a dodici mesi è costruire un sistema in grado di:

- raccogliere in modo continuo e a basso overhead segnali kernel e scheduler tramite eBPF;
- classificare automaticamente i workload in classi operative ben definite;
- selezionare e applicare politiche scheduler ottimali tramite sched-ext e cgroup v2;
- validare l'impatto delle ottimizzazioni con una feedback loop controllata e rollback automatico in caso di regressione;
- esportare metriche e insight tramite OpenTelemetry, integrabili con dashboard e CLI esistenti.

Il posizionamento volutamente evita la sovrapposizione con piattaforme di ottimizzazione autonoma full-stack come Akamas. CortexBrain non propone tuning black-box di Kubernetes o runtime applicativi, ma piuttosto un livello di ottimizzazione più profondo: lo scheduler del kernel Linux, pluggable tramite sched-ext, guidato da dati reali di workload.

2. Contesto tecnologico

2.1 Lo shift verso AI e HPC su bare metal

L'adozione di workload AI e HPC sta uscendo rapidamente dal modello cloud-only. Molte organizzazioni, per motivi di costo, controllo, latenza e sovranità dei dati, stanno ricollocando questi carichi su infrastrutture on-premise, bare metal ed edge. Questo shift crea nuove esigenze:

- tuning fine-grained delle risorse fisiche (CPU, NUMA, cache, memory bandwidth);
- controllo della latenza in workload real-time e inference;
- massimizzazione del throughput in training e simulazioni HPC;
- operatività in ambienti con risorse limitate, dove il margine di spreco è minimo.

In questo scenario il kernel Linux diventa un livello critico. Le decisioni dello scheduler influenzano direttamente prestazioni, determinismo e utilizzo delle risorse.

2.2 sched-ext: lo scheduler Linux come componente pluggable

sched-ext, introdotto nel kernel Linux 6.12, permette di implementare classi scheduler come programmi eBPF caricabili a runtime. Rispetto alla scrittura di un nuovo scheduler nel kernel, sched-ext offre:

- iterazione rapida senza ricompilare il kernel;
- sicurezza garantita dal verifier eBPF;
- possibilità di caricare, scaricare e confrontare scheduler in produzione;
- accesso a strutture kernel controllate tramite API stabilite.

Per CortexBrain, sched-ext rappresenta l'enabler tecnologico principale: permette di tradurre le metriche raccolte in politiche scheduler concrete e applicabili.

2.3 eBPF come standard per osservabilità kernel a basso overhead

eBPF è ormai lo standard de facto per tracciare eventi kernel in produzione. Le principali piattaforme di osservabilità (Cilium, Falco, Tetragon, Polar Signals, Datadog, Netflix, Meta) lo utilizzano per networking, sicurezza e profiling. Per l'uso in ottimizzazione scheduler, eBPF consente:

- attaccarsi a tracepoint scheduler senza modificare il kernel;
- aggregare dati in mappe BPF, riducendo drasticamente il traffico verso user-space;
- correlare eventi per PID, TGID, CPU, NUMA node;
- misurare con precisione tempi kernel in nanosecondi.

L'adozione di Aya come framework Rust per eBPF mantiene il codebase coerente con l'architettura esistente di CortexBrain.

2.4 Requisiti kernel e ambiente

Il sistema target è Linux kernel 6.12 o superiore con le seguenti opzioni abilitate:

- CONFIG_BPF
- CONFIG_BPF_SYSCALL
- CONFIG_BPF_JIT
- CONFIG_TRACEPOINTS
- CONFIG_SCHED_CLASS_EXT
- CONFIG_DEBUG_INFO_BT

L'ambiente di riferimento è bare-metal edge, non virtualizzato, con workload containerizzati o nativi. Non è previsto supporto per kernel legacy o ambienti cloud multi-tenant in questa fase.

3. Visione architetturale

L'architettura target di CortexBrain si articola in cinque livelli:

3.1 Data Collection Layer (eBPF kernel-space)

Questo livello contiene i programmi eBPF che tracciano eventi kernel e scheduler. I programmi principali saranno:

- tracepoint su sched:sched_switch, sched:sched_wakeup, sched:sched_stat_runtime, sched:sched_stat_wait, sched:sched_migrate_task;
- tracepoint su kmem:kmalloc e kmem:kfree;
- tracepoint su syscalls:sys_enter_mmap e syscalls:sys_exit_mmap;
- perf events hardware per cycles, instructions, cache misses, correlati per TGID;
- eventuali probe su funzioni sched-ext per esporre metriche custom.

3.2 BPF Maps (aggregazione in-kernel)

Per garantire basso overhead, i programmi eBPF non emetteranno ogni evento verso user-space. Invece, aggiorneranno mappe BPF di aggregazione:

- sched_stats: per-TGID statistica scheduler (wait time, run time, context switches, migrations);
- kmem_stats: per-TGID allocazioni e deallocazioni kernel heap;
- workload_class: mapping TGID -> classe workload;
- perf_hw_stats: per-TGID contatori hardware performance events.

Le mappe verranno lette periodicamente dall'agent user-space (es. ogni secondo o ogni 5 secondi).

3.3 User-space Agent (Rust)

L'agent, estendendo il componente `metrics` esistente, si occupa di:

- caricare e gestire i programmi eBPF tramite Aya;
- leggere le mappe BPF a intervalli regolari;
- classificare i workload;
- alimentare l'Optimization Engine;
- esportare metriche OpenTelemetry.

3.4 Workload Classifier

Modulo Rust che, in base ai segnali raccolti, assegna ogni TGID a una classe workload. Il classificatore è deterministico e basato su regole esplicite, non su modelli black-box.

3.5 Scheduler Optimization Engine

Il cuore decisionale del sistema. Riceve in input la classe workload e le metriche attuali, e produce in output:

- selezione dello scheduler `sched-ext` da caricare;
- parametri dello scheduler;
- configurazioni cgroup v2 (`cpu.weight`, `cpu.uclamp.min/max`, `cpuset.cpus`, `cpuset.mems`);
- eventuali `sysctl` controllate.

3.6 sched-ext BPF Scheduler

Una o più implementazioni di scheduler eBPF caricate a seconda della classe workload dominante. Ogni scheduler implementa una politica specifica:

- AI training: affinity su core fisici, riduzione migration, priorità throughput;
- AI inference: low-latency dispatch, wake-up rapido, predizione burst;
- HPC batch: fairness controllata, minimizzazione context switch;
- HPC real-time: determinismo, deadline awareness, isolation;
- I/O-bound: rispetto ai tempi di I/O wait;
- mixed: politica adattiva basata su sottoclassi.

3.7 Feedback Loop e Rollback

Dopo ogni modifica applicata, il sistema continua a monitorare le metriche chiave. Se rileva regressione rispetto a una baseline, può:

- segnalare l'anomalia;
- tornare alla configurazione precedente;
- registrare l'esperimento in un log strutturato per analisi successiva.

L'approccio è controllato: le ottimizzazioni non sono applicate in modo autonomo aggressivo, ma con garanzia di reversibilità.

3.8 OpenTelemetry Exporter e Dashboard

Le metriche aggregate vengono esportate tramite OTLP gRPC/HTTP. Il componente `otel_metrics` esistente verrà esteso con nuovi strumenti. La dashboard visualizzerà:

- distribuzione dei workload per classe;
 - metriche scheduler per TGID e per classe;
 - allocazioni kernel heap;
 - eventi di tuning e risultati della feedback loop.
-

4. Stato attuale dei componenti CortexBrain

4.1 Componente metrics_tracer

Il programma eBPF metrics_tracer è il principale raccoglitore di eventi kernel. I programmi attualmente caricati sono:

- metrics_tracer: kprobe su tcp_identify_packet_loss, raccoglie metriche socket;
- tcp_v4_connect / tcp_v6_connect: kprobe per tracciare l'inizio connessione TCP;
- tcp_rcv_state_process: kprobe per calcolare la latenza handshake TCP;
- trace_cpu_frequency: tracepoint su percpu:percpu_alloc_percpu, raccoglie bytes_alloc e TGID;
- trace_enter_mmap: tracepoint su syscalls:sys_enter_mmap, raccoglie indirizzo, lunghezza e TGID.

I dati vengono inviati a user-space tramite PerfEventArray.

4.2 Strutture dati esistenti

Le strutture C-compatibili attualmente definite sono:

- NetworkMetrics: metriche socket (sk_err, sk_drops, backlog, write memory queued, receive buffer, ack backlog);
- TimestampStartInfo e TimestampEvent: correlazione connect -> SYN-ACK, con delta in microsecondi;
- CpuFrequency: bytes_alloc per CPU con PID/TGID e command;
- MemAlloc: mmap length, addr, tgid, command.

Le mappe BPF esistenti sono TIME_STAMP_START, TIME_STAMP_EVENTS, NET_METRICS, CPU_FREQUENCY, MEM_ALLOC.

4.3 Metriche OpenTelemetry attuali

Nel modulo otel_metrics.rs sono definiti i seguenti strumenti:

- events_total: counter generico;
- packets_total: counter eventi network;
- sk_drops, sk_err: gauge socket;
- delta_us, ts_us: histogram timestamp;
- cpu_bytes_alloc_events_total: counter allocazioni percpu;
- cpu_bytes_alloc: gauge bytes allocati percpu;
- mem_alloc_events_total: counter mmap;
- enter_mem_alloc: gauge bytes richiesti via mmap.

4.4 Correlazione workload

La correlazione è attualmente limitata a **TGID** e **command name**. Non è presente correlazione per container, cgroup, namespace o K8s pod. Questo è sufficiente per il primo MVP, ma dovrà essere esteso in futuro per ambienti orchestrati.

5. Piano di evoluzione delle metriche

5.1 Metriche da mantenere e rafforzare

cpu_bytes_alloc (percpu_alloc_percpu)

- Utilità: profilare l'allocazione della memoria per-CPU del kernel per workload.
- Azione: mantenere, rinominare in kernel_percpu_alloc_bytes.

- Aggiunta consigliata: conteggio eventi, dimensione media allocazione.

enter_mem_alloc (mmap)

- Utilità: capire il pattern di allocazione userspace (AI/HPC fanno uso intensivo di mmap).
- Azione: mantenere, aggiungere sys_exit_mmap per correlare richiesta e allocazione effettiva.
- Aggiunta consigliata: lifetime delle regioni mmap.

Latenza handshake TCP (delta_us)

- Utilità: indicatore indiretto per workload latency-sensitive.
- Azione: mantenere, rinominare in tcp_handshake_latency_us.

5.2 Metriche da riprogettare o spostare

events_total

- Problema: troppo generico, poco azionabile.
- Azione: sostituire con counter per categoria: scheduler_events_total, memory_events_total, network_events_total.

Metriche socket (sk_drops, sk_err, sk_backlog_len, sk_write_memory_queued, ecc.)

- Problema: utili ma non correlate al tuning scheduler.
- Azione: spostare in un componente network separato (network_tracer), tenendo metrics_tracer focalizzato su CPU/memory/scheduler.

ts_us

- Problema: timestamp interno, non è una metrica di business.
- Azione: eliminare dagli strumenti OTel, conservarlo solo come campo dati grezzo.

5.3 Metriche da aggiungere

La seguente tabella riassume le metriche da introdurre, organizzate per area.

Area: Scheduler latency e behavior

Nome metrica	Tipo OTel	Fonte eBPF	Priorità
scheduler_wait_time_us	Histogram	sched:sched_stat_wait, sched:sched_wakeup	Alta
scheduler_run_time_us	Histogram	sched:sched_stat_runtime	Alta
scheduler_ctx_switches_total	Counter	sched:sched_switch	Alta
scheduler_wakeup_latency_us	Histogram	sched:sched_wakeup vs sched_switch	Media
scheduler_migrations_total	Counter	sched:sched_migrate_task	Media
scheduler_runqueue_latency_us	Histogram	derivata da wait + switch	Alta

Area: CPU e Runqueue

Nome metrica	Tipo OTel	Fonte eBPF	Priorità
cpu_runnable_time_us	Gauge	sched:sched_stat_wait per CPU	Alta
cpu_running_time_us	Gauge	sched:sched_stat_runtime per CPU	Alta
cpu_nr_running	Gauge	/proc/schedstat o kprobe su runqueue	Media

cpu_idle_state_changes_total	Counter	power:cpu_idle	Bassa
------------------------------	---------	----------------	-------

Area: NUMA e località memoria

Nome metrica	Tipo OTel	Fonte eBPF	Priorità
numa_migrations_total	Counter	sched:sched_migrate_task con src/dst node	Media
numa_local_page_alloc_total	Counter	kmem:mm_page_alloc con node locale	Media
numa_remote_page_alloc_total	Counter	kmem:mm_page_alloc con node remoto	Media

Area: Kernel heap allocations

Nome metrica	Tipo OTel	Fonte eBPF	Priorità
kernel_kmalloc_bytes_total	Counter	kmem:kmalloc aggregato per TGID	Alta
kernel_kmalloc_count_total	Counter	kmem:kmalloc count per TGID	Alta
kernel_kfree_bytes_total	Counter	kmem:kfree per TGID	Media
kernel_kmalloc_live_bytes	Gauge	kmalloc - kfree per TGID	Alta
kernel_kmalloc_avg_size_bytes	Gauge	derivata da count e bytes	Media

Area: Hardware performance events

Nome metrica	Tipo OTel	Fonte eBPF	Priorità
cpu_cycles_total	Counter	perf event hardware	Alta
cpu_instructions_total	Counter	perf event hardware	Alta
cpu_cache_misses_total	Counter	perf event hardware	Alta
cpu_cache_references_total	Counter	perf event hardware	Media
cpi_ratio	Gauge	cycles / instructions	Alta

Area: sched-ext specific

Nome metrica	Tipo OTel	Fonte eBPF	Priorità
scx_dispatch_latency_us	Histogram	custom probe nello scheduler eBPF	Media
scx_queue_depth	Gauge	custom probe nello scheduler eBPF	Media
scx_task_class	Gauge	mapping TGID -> classe	Alta
scx_policy_active	Gauge	stato scheduler caricato	Alta

5.4 Principio di raccolta

Per rispettare il vincolo di basso overhead:

- gli eventi scheduler e kmalloc non verranno inviati uno per uno a user-space;
- verranno aggregati in mappe BPF con chiave TGID e, dove rilevante, CPU/NUMA node;
- l'agent user-space leggerà le mappe a intervalli regolari (1-5 secondi);
- i perf buffer saranno riservati solo a eventi rari o di segnalazione (es. cambio classe workload).

6. Workload classification

6.1 Classi workload target

Il sistema classificherà i workload nelle seguenti classi:

- **ai_training**: alto utilizzo CPU/GPU, pattern batch, accesso memory-bound, poche context switch, sensibile a throughput.
- **ai_inference**: burst di richieste, latenza critica, pattern di allocazione ripetitivo, elevato cache pressure.
- **hpc_batch**: lunghi job CPU-bound, pattern regolare, parallelismo, sensibile a fairness e throughput.
- **hpc_realtime**: vincoli temporali, bassa latenza, determinismo, isolamento.
- **latency_sensitive**: servizi interattivi, bassa latenza di risposta, molti wakeup.
- **throughput_bound**: data processing, ingestion, pipeline, obiettivo massimo throughput.
- **io_bound**: tempo dominato da attesa I/O, poco CPU-intensive.
- **mixed**: pattern eterogeneo, richiede adattamento dinamico.

6.2 Segnali di classificazione

Ogni classe viene riconosciuta combinando i seguenti segnali:

- **CPU intensity**: cycles per second, instructions per second;
- **Memory intensity**: mmap rate, kmalloc rate, cache miss rate;
- **Scheduler pattern**: wait/run ratio, context switch frequency, wakeup frequency;
- **I/O pattern**: tempo in sleep/blocked, se disponibile;
- **Duration behavior**: job lunghi vs burst brevi;
- **Parallelismo**: numero di thread, distribuzione su CPU.

6.3 Esempi di mapping iniziale

Classe	Segnali dominanti
ai_training	CPI alto, mmap grandi, poche context switch, run time lungo
ai_inference	burst di wakeup, latenza scheduler bassa, cache miss medio-alto
hpc_batch	CPI basso, runtime lungo, migrazioni rare
hpc_realtime	wait time molto basso, jitter minimo, isolamento richiesto
latency_sensitive	molti wakeup, run time breve, coda ready contenuta
throughput_bound	run time medio-lungo, context switch moderate, CPU alta
io_bound	sleep time alto, run time basso, context switch legate a I/O
mixed	varianza elevata nei segnali nel tempo

6.4 Architettura del classificatore

Il classificatore è implementato in Rust come modulo deterministico. Per ogni TGID attivo, periodicamente:

1. legge i valori aggregati dalle mappe BPF;
2. normalizza i segnali;
3. applica regole di decisione in cascata;
4. assegna una classe;
5. aggiorna la mappa BPF `workload_class`;

6. notifica l'Optimization Engine se la classe cambia.

Il classificatore non utilizza machine learning black-box. Le regole sono esplicite, revisionabili e versionabili.

7. Scheduler Optimization Engine

7.1 Filosofia

L'Optimization Engine opera con politiche deterministiche, esplicite e reversibili. Non utilizza reinforcement learning autonomo black-box, per evitare sovrapposizioni con brevetti esistenti nel mercato e per garantire trasparenza.

7.2 Input

- classe workload per TGID;
- metriche scheduler attuali;
- metriche di allocazione kernel;
- obiettivo di ottimizzazione (latency, throughput, determinismo, fairness);
- vincoli di sistema (core disponibili, topology NUMA, carico attuale).

7.3 Output

- scelta dello scheduler sched-ext da caricare;
- parametri dello scheduler;
- configurazioni cgroup v2;
- eventuali sysctl;
- ordine di applicazione e rollback plan.

7.4 Mapping classe -> politica iniziale

Classe	Politica scheduler suggerita
ai_training	Scheduler con core pinning, riduzione migration, time slice elevato
ai_inference	Scheduler low-latency dispatch, wake-up priority, time slice breve
hpc_batch	FIFO/Round-robin con fairness, minimizzazione context switch
hpc_realtime	Scheduler deadline-aware o partitioned, isolamento core dedicati
latency_sensitive	Priority boosting, latency-aware preemption
throughput_bound	Batch-friendly, time slice lungo, fairness controllata
io_bound	Ottimizzazione wake-up, evitare busy-waiting
mixed	Scheduler adattivo con sottocoda per classe

7.5 Feedback loop

Dopo ogni tuning:

1. registra baseline (5-15 minuti prima dell'intervento);
2. applica la modifica;
3. monitora per 5-30 minuti;
4. confronta metriche chiave rispetto alla baseline;
5. se miglioramento $\geq$ soglia: conferma;
6. se regressione $\geq$ soglia: rollback automatico;

7. se neutro: registra e continua monitoraggio.

La soglia di regressione/miglioramento è configurabile per metrica.

8. sched-ext BPF Scheduler

8.1 Requisiti generali

Gli scheduler sched-ext di CortexBrain devono:

- essere scritti in eBPF/C e compilati tramite toolchain sched-ext;
- esporre metriche interne tramite mappe BPF;
- supportare il caricamento e lo scaricamento a runtime;
- rispettare i vincoli del verifier eBPF.

8.2 Scheduler base

Il primo scheduler da implementare è uno scheduler **workload-aware adattivo** con le seguenti caratteristiche:

- più runqueue per classe workload;
- dispatch basato su priorità di classe;
- meccanismo di load balancing controllato;
- possibilità di core pinning per classi HPC/RT.

8.3 Scheduler specializzati (futuri)

Successivamente si possono implementare scheduler dedicati:

- `scx_ai_training`: throughput-oriented, affinity-based;
- `scx_ai_inference`: latency-oriented, wake-up optimized;
- `scx_hpc_realtime`: partitioned, deadline-aware;
- `scx_io_optimized`: wake-up coalescing, I/O-friendly.

8.4 Interazione con Optimization Engine

L'Optimization Engine sceglie lo scheduler e i parametri. Lo scheduler eBPF legge la mappa `workload_class` per conoscere la classe di ogni task e applicare la politica corrispondente.

9. Roadmap di implementazione

Fase 1: Raccolta dati scheduler e kernel heap (4-6 settimane)

Obiettivo: aggiungere i tracepoint fondamentali e l'aggregazione in mappe BPF.

Attività:

- aggiungere tracepoint sched in `metrics_tracer`;
- aggiungere tracepoint `kmem:kmalloc` e `kmem:kfree`;
- creare mappe BPF `sched_stats` e `kmem_stats`;
- aggiornare l'agent user-space per leggere le nuove mappe;
- estendere `otel_metrics.rs` con nuovi strumenti.

Output:

- metriche scheduler e kernel heap esportate in OpenTelemetry.

Fase 2: Workload classification (3-4 settimane)

Obiettivo: classificare i workload in classi target.

Attività:

- definire struttura dati per classe workload;
- implementare classificatore deterministico in Rust;
- aggiungere mappa BPF workload_class;
- esporre metrica scx_task_class;
- validare classificazione su workload sintetici.

Output:

- mapping TGID -> classe workload consultabile e visibile in dashboard.

Fase 3: Integrazione sched-ext (4-6 settimane)

Obiettivo: caricare e gestire scheduler sched-ext workload-aware.

Attività:

- integrare nel caricamento programmi sched-ext in Aya;
- implementare scheduler base adattivo;
- aggiungere metriche interne allo scheduler;
- testare su workload sintetici;
- gestire caricamento/scaricamento sicuro.

Output:

- sistema in grado di caricare uno scheduler sched-ext basato sulla classe workload.

Fase 4: Optimization Engine e feedback loop (4-6 settimane)

Obiettivo: decisioni di tuning controllate e reversibili.

Attività:

- implementare policy engine con mapping classe -> politica;
- aggiungere tuning cgroup v2;
- implementare baseline, confronto, rollback;
- logging strutturato degli esperimenti;
- test A/B controllati.

Output:

- primo tuning automatico con garanzia di rollback.

Fase 5: Integrazione prodotto (3-4 settimane)

Obiettivo: rendere la funzionalità fruibile tramite CLI e dashboard.

Attività:

- aggiungere comandi CLI:
 - cfcli workload classify
 - cfcli scheduler status
 - cfcli scheduler tune --workload <name>
 - cfcli scheduler history
- aggiornare dashboard OTel con nuove viste;
- documentare flusso operativo;
- preparare release notes.

Output:

- feature rilasciata come componente del prodotto CortexBrain.
-

10. Requisiti e vincoli tecnici

10.1 Kernel

- Linux 6.12 o superiore;
- CONFIG_SCHED_CLASS_EXT=y;
- BTF abilitato;
- tracepoint sched e kmem disponibili.

10.2 Toolchain

- Rust stable (versione compatibile con Aya);
- Aya e aya-tool;
- sched-ext toolchain (scx_utils, sched_ext API);
- OpenTelemetry Rust SDK;
- BPF toolchain (clang, llvm).

10.3 Ambiente

- Bare-metal edge inizialmente;
- Possibile estensione a VM cloud con kernel custom in futuro;
- Non supportato su kernel legacy (< 6.12).

10.4 Overhead

- Target overhead del tracciamento eBPF: inferiore al 3% su workload rappresentativi;
 - Aggregazione in-kernel obbligatoria per eventi ad alta frequenza (scheduler, kmalloc);
 - Perf buffer solo per eventi rari e segnalazioni.
-

11. Posizionamento di mercato e differenziazione

11.1 Mercato italiano

In Italia esistono settori con forte domanda di performance kernel-level:

- automotive: simulazione, ADAS, testing;
- telco: 5G edge, low latency, NFV;
- energy: edge computing, SCADA, real-time;
- manufacturing: linee di produzione, robotica, visione artificiale;
- ricerca HPC: università, centri supercomputing, consorzi.

Akamas, azienda italiana, copre principalmente ottimizzazione full-stack Kubernetes/JVM/cloud. CortexBrain non deve competere su quel territorio.

11.2 Differenziazione rispetto ad Akamas

Aspetto	Akamas	CortexBrain target
Livello di ottimizzazione	Applicazione, runtime, K8s, cloud	Kernel Linux scheduler
Tecnologia core	RL brevettato, full-stack tuning	eBPF + sched-ext, deterministico
Ambiente	Cloud, Kubernetes, JVM	Bare-metal edge, AI/HPC/RT
Approccio	Autonomous black-box	Engineering-driven, esplicitabile

Deliverable	Raccomandazioni / auto-apply	Scheduler pluggable, policy, metriche
-------------	------------------------------	---------------------------------------

11.3 Gap di mercato

Il mercato ha molti tool di osservabilità eBPF e molte piattaforme di ottimizzazione cloud. Manca invece un prodotto open source che:

- combini osservabilità eBPF con azione kernel-level tramite sched-ext;
- classifichi workload in modo deterministico per scopi scheduler;
- offra scheduler specializzati per AI/HPC/RT su bare metal.

CortexBrain può colmare questo gap.

11.4 Value proposition di sintesi

CortexBrain è la piattaforma open source per l'ottimizzazione dello scheduler Linux su workload AI, HPC e real-time. Tramite eBPF e sched-ext, profila il comportamento dei workload a livello kernel e applica politiche scheduler data-driven, esplicibili e reversibili, su infrastrutture bare-metal ed edge.

12. Considerazioni su brevetti e proprietà intellettuale

12.1 Contesto

Akamas dichiara pubblicamente di detenere un portafoglio di brevetti su tecnologie di ottimizzazione autonoma basate su reinforcement learning. Il loro focus è l'ottimizzazione full-stack mediante algoritmi di ottimizzazione automatica.

12.2 Strategia di CortexBrain

Per evitare sovrapposizioni con brevetti esistenti:

- non utilizzare reinforcement learning black-box per il tuning;
- non posizionare il prodotto come "autonomous optimization platform" generica;
- focalizzarsi su un dominio tecnico specifico: lo scheduler Linux pluggable tramite sched-ext;
- mantenere il decisionale basato su regole esplicite, engineering-driven;
- pubblicare il core come open source, favorendo trasparenza e review comunitaria.

12.3 Raccomandazione

Evitare completamente l'adozione di tecniche di machine learning black-box per decisioni autonome di ottimizzazione. Il valore competitivo risiede nella profondità kernel, nella bassa overhead, nella trasparenza delle politiche e nell'integrazione con sched-ext, non in un algoritmo di ottimizzazione generico brevettabile.

13. Modello di business: suggerimento

Non essendo richiesto un capitolo approfondito, si suggerisce il seguente modello:

- **Core open source** sotto licenza Apache 2.0, coerentemente con il repository attuale;
- **Professional services**: consulenza, deployment, tuning su workload specifici;
- **Enterprise support**: supporto tecnico, SLAs, backport su kernel supportati;

- **Dual license / feature avanzate** eventualmente valutabile in futuro per componenti non-core (es. scheduler proprietari verticali).

Questo modello consente di costruire community, credibilità tecnica e pipeline commerciale senza competere direttamente con piattaforme SaaS di ottimizzazione autonoma.

14. Rischi e mitigazioni

14.1 Mercato ristretto

- Rischio: il tuning scheduler è nicchia molto tecnica.
- Mitigazione: partire da casi d'uso verticali (AI inference su edge, HPC batch) dove il valore è misurabile.

14.2 Adozione lenta del kernel moderno

- Rischio: molte aziende non aggiornano rapidamente il kernel.
- Mitigazione: targettare inizialmente ambienti bare-metal edge e HPC dove il controllo del kernel è maggiore.

14.3 ROI difficile da dimostrare

- Rischio: migliorare la latenza dello scheduler del 5% è meno vendibile di ridurre il cloud cost.
- Mitigazione: costruire benchmark pubblici e case studies quantificati su workload reali.

14.4 Complessità tecnica

- Rischio: sched-ext, eBPF, kernel internals richiedono competenze rare.
- Mitigazione: investire in documentazione, esempi, test automatizzati e community.

14.5 Concorrenza

- Rischio: grandi player possono entrare nello spazio sched-ext.
 - Mitigazione: muoversi rapidamente, costruire verticali specifici, mantenere open source per community lock-in.
-

15. KPI e metriche di successo

15.1 Adozione e community

- numero di star/fork/contributor su GitHub;
- download del CLI;
- partecipazione a issue e discussioni.

15.2 Validazione tecnica

- riduzione latenza scheduler per workload target;
- aumento throughput in benchmark HPC/AI;
- riduzione migrazioni NUMA;
- overhead del tracciamento sotto il 3%;
- stabilità in esecuzione prolungata (uptime, crash).

15.3 Prodotto

- numero di classi workload riconosciute correttamente;
- numero di tuning applicati con esito positivo;
- tempo medio di rollback in caso di regressione;

- copertura metriche OpenTelemetry.
-

16. Prossimi passi immediati

1. **Avviare Fase 1:** aggiungere tracepoint scheduler e kalloc in `metrics_tracer` con aggregazione in mappe BPF.
 2. **Validare overhead:** misurare impatto del nuovo tracciamento su workload sintetici.
 3. **Definire scheduler base sched-ext:** preparare ambiente di sviluppo e primo scheduler workload-aware.
 4. **Mantenere focus:** non espandere il progetto verso ottimizzazione K8s/config generica; restare su kernel scheduler.
 5. **Documentare progressi:** aggiornare questa guida ogni 4-6 settimane con stato avanzamento e decisioni prese.
-

17. Evoluzione verso sistemi micro-ottimizzati

17.1 Visione di medio-lungo termine

Dopo il primo anno, l'obiettivo di CortexBrain non è più solo ottimizzare lo scheduler di un kernel Linux generico, ma evolvere verso la costruzione di **sistemi micro-ottimizzati per workload specifici**. Il kernel viene considerato come un sistema software configurabile e specializzabile: ogni deployment può ricevere un profilo ottimale per la classe di carico dominante.

Questa evoluzione si articola in due fasi:

- **Fase D (Year 2):** specializzazione di un kernel Linux minimale tramite eBPF, sched-ext e configurazioni controllate. Questo è l'estensione naturale del lavoro del primo anno.
- **Fase B (Year 3+):** esplorazione di microkernel veri e propri come target architetturale alternativo, qualora il percorso Linux+eBPF mostri limiti di isolamento, determinismo o dimensionamento.

17.2 Da scheduler tuning a micro-specializzazione

Nel primo anno il sistema raccoglie dati, classifica workload e seleziona scheduler. Nel secondo anno lo stesso approccio viene esteso a più sottosistemi del kernel, producendo un **CortexBrain Micro-OS Profile**: una configurazione coerente di scheduler, memory, network, storage, interrupt e boot per un workload target.

Il profilo non è un microkernel nel senso classico, ma un'istanza di Linux fortemente specializzata, guidata da dati e generata con assistenza AI, sotto controllo umano.

17.3 Principi guida

- **Specializzazione senza fork:** non si creano fork di Linux, ma profili applicabili a kernel standard.
 - **eBPF come meccanismo di estensione:** programmi eBPF sostituiscono o affiancano comportamenti kernel dove possibile.
 - **Human-in-the-loop:** l'AI genera profili e componenti, ma la validazione, la build e il deployment rimangono sotto controllo umano.
 - **Reversibilità:** ogni profilo può essere scaricato e il sistema può tornare alla configurazione precedente.
 - **Misurabilità:** ogni profilo viene validato tramite benchmark prima del deployment in produzione.
-

18. CortexBrain Micro-OS Profile (Year 2)

18.1 Definizione

Un **Micro-OS Profile** è un pacchetto strutturato che contiene:

- una selezione di programmi eBPF (scheduler, network, tracing);
- uno o più scheduler sched-ext;
- una configurazione kernel minimale (Kconfig);
- un set di parametri cgroup v2 e sysctl;
- una configurazione di boot/init (servizi minimi, CPU isolation, hugepages);
- metadati di classificazione workload;
- una pipeline di validazione e benchmark.

Il profilo viene generato dall'AI assistant a partire dalle metriche osservate, revisionato dall'ingegnere e applicato tramite la piattaforma CortexBrain.

18.2 Componenti di un profilo

Kernel configuration

- Kconfig minimale con solo i sottosistemi necessari al workload.
- Driver essenziali, network stack ridotto o meno, filesystem selezionato.
- Opzioni per real-time, preempt, tickless, NO_HZ_FULL se richiesto.

eBPF programs

- Scheduler sched-ext workload-aware.
- Programmi XDP/TC per network stack ridotto o accelerato.
- Tracepoint per monitoraggio continuo.
- Programmi di sicurezza/integrità opzionali.

Resource configuration

- cgroup v2: cpu, cpuset, memory, io.
- sysctl: kernel scheduler tunables, VM parameters, network buffers.
- Hugepages, NUMA binding, IRQ affinity, CPU isolation.

Boot/init

- Init system minimale (es. systemd con unit ridotte, o init custom).
- CPU isolation, isolcpus o cpuset per workload critico.
- Servizi non essenziali disabilitati.

18.3 AI-assisted profile generation

L'AI, osservando un workload reale, propone la struttura del profilo:

1. analizza le metriche aggregate (scheduler, memory, network, I/O);
2. seleziona la classe workload dominante;
3. sceglie un template base umano predefinito;
4. adatta parametri e componenti in base ai segnali;
5. genera una bozza di profilo con spiegazioni;
6. l'ingegnere revisiona e corregge;
7. il profilo viene validato in benchmark;
8. dopo successo, viene marcato come pronto per produzione.

L'AI non genera codice kernel arbitrario: opera entro uno spazio di template e parametri controllati, definito dagli ingegneri.

18.4 Esempi di profili target

Classe workload	Focus ottimizzazione	Componenti chiave
ai_training	Throughput CPU/GPU, memory bandwidth, minimizzazione jitter	sched-ext affinity-based, hugepages 1G, NUMA binding, RPS/XPS ottimizzato
ai_inference	Latenza burst, wake-up rapido, cache locality	sched-ext low-latency, busy polling network, tickless su core critici
hpc_batch	Fairness, long time slice, riduzione context switch	sched-ext FIFO-like, isolcpus, I/O scheduler mq-deadline o none
hpc_realtime	Determinismo, deadline awareness, isolation	PREEMPT_RT o configurazione real-time, core pinning, watchdog
io_bound	Efficienza I/O, minimizzazione busy-wait	I/O scheduler tuning, async I/O, network coalescing

18.5 Pipeline di build e validazione

La generazione di un profilo segue una pipeline rigida:

1. **Data collection:** metriche del workload target raccolte per un periodo rappresentativo.
 2. **Profile synthesis:** AI genera bozza di profilo.
 3. **Human review:** ingegnere valida scelte e parametri.
 4. **Build:** toolchain produce immagine kernel + eBPF programs + init.
 5. **Simulation/test:** test in VM o hardware rappresentativo.
 6. **Benchmark:** confronto A/B con kernel generico.
 7. **Staging deployment:** profilo applicato in ambiente non produttivo.
 8. **Production deployment** con monitoraggio continuo.
 9. **Feedback loop:** se metriche peggiorano, rollback o rigenerazione.
-

19. Ottimizzazioni oltre lo scheduler

19.1 Network stack

Area	Tecnica	Scopo
XDP	Programma eBPF su RX path per drop/redirect veloce	Riduzione latenza network per workload latency-sensitive
TC	Classificazione e shaping del traffico in kernel	Priorizzazione traffico per classe workload
RPS/XPS	Affinità RX/TX per CPU	Località cache, riduzione interrupt migration
Busy polling	Eliminare interrupt-driven RX per alto throughput	Massimizzare throughput pacchetti
IRQ affinity	Associare interrupt a core specifici	Isolare workload critico da interrupt

19.2 Memory subsystem

Area	Tecnica	Scopo
------	---------	-------

Hugepages	1G/2M hugepages per workload memory-bound	Riduzione TLB miss, miglioramento bandwidth
NUMA policy	Binding memoria a node locale	Località, riduzione latenza memoria
cgroup memory	Limiti e priorità per workload	Isolamento, prevenzione memory pressure
Swappiness	Tuning per workload	Evitare swap in workload RT, permetterlo in batch
Transparent hugepages	Abilitare/disabilitare in base al workload	Trade-off tra latenza e throughput

19.3 Storage e I/O

Area	Tecnica	Scopo
I/O scheduler	none/mq-deadline/kyber/bfq per workload	Minimizzare latenza o massimizzare throughput
Readahead	Tuning page cache readahead	Migliorare pattern di lettura sequenziale
Async I/O	io_uring, AIO	Parallelismo I/O per workload data-intensive
Block layer	Multi-queue, queue depth, polling	Throughput storage

19.4 Boot e CPU isolation

Area	Tecnica	Scopo
isolcpus	Isolare core dal scheduler generico	Dedicare core a workload critico
NO_HZ_FULL	Disabilitare timer tick su core isolati	Ridurre jitter su core RT
RCU offloading	Spostare RCU su core non critici	Minimizzare interruzioni su core workload
Servizi minimi	Disabilitare unit systemd non necessarie	Ridurre footprint e interferenze
Init system	Init custom o systemd minimale	Boot rapido e deterministico

20. Microkernel research track (Year 3+)

20.1 Motivazione

Se il percorso Linux+eBPF mostrerà limiti in termini di:

- dimensione e complessità del kernel;
- determinismo e latenza garantita;
- isolamento tra componenti;
- sicurezza e verificabilità;

allora CortexBrain potrà valutare l'adozione o lo sviluppo di un microkernel vero.

20.2 Candidati tecnologici

Microkernel	Caratteristiche	Rilevanza per CortexBrain
-------------	-----------------	---------------------------

seL4	Formalmente verificato, capability-based, altamente sicuro	Adatto a workload RT/safety-critical, ma ecosistema limitato
L4Re	Framework per sistemi basati su L4	Maggiore ecosistema rispetto a seL4, adatto a micro-OS custom
Fuchsia/Zircon	Microkernel moderno, capability-based, usato da Google	Ecosistema in crescita, ma controllato da Google
Unikraft	Unikernel framework, specializzabile per applicazione	Adatto a edge/IoT, meno a HPC general purpose
Rux/Theseus	Sistemi OS in Rust, ricerca attiva	Interessante per sicurezza memory-safe, ma immaturo

20.3 Come portare policy AI su microkernel

In un microkernel, la logica di scheduling e resource management risiede in gran parte in user-space. L'AI può:

- generare policy di scheduling per i server in user-space;
- produrre configurazioni di capability e IPC;
- ottimizzare il mapping di task/driver su domini di protezione;
- definire policy di memory partitioning.

A differenza di Linux+eBPF, qui l'AI genera componenti user-space, non estensioni kernel. Il controllo umano resta centrale.

20.4 Criteri di decisione per il passaggio a microkernel

La valutazione di un microkernel vero verrà fatta solo se:

- esistono casi d'uso con requisiti di isolamento/sicurezza non soddisfacibili con Linux;
- il team ha acquisito competenze sufficienti su microkernel;
- esistono clienti o progetti pilota disposti a sperimentare;
- il costo di manutenzione è giustificato dal valore aggiunto.

Fino a quel momento, il focus resta su Linux+eBPF come piattaforma pratica e adottabile.

21. Nuove metriche per Year 2

Oltre alle metriche scheduler, memory e hardware del primo anno, il secondo anno introduce:

Nome metrica	Tipo OTel	Fonte	Priorità
network_latency_us	Histogram	XDP/TC/tracepoint socket	Alta
network_throughput_bytes_total	Counter	XDP/TC	Alta
io_wait_time_us	Histogram	sched:sched_stat_iowait	Alta
io_throughput_bytes_total	Counter	block tracepoint	Media
memory_hugepages_used	Gauge	proc fs / cgroup	Media
boot_time_seconds	Gauge	Misura boot	Bassa
irq_count_per_cpu	Counter	irq tracepoint	Media
thermal_throttle_count	Counter	thermal tracepoint	Bassa

22. KPI aggiornati e visione di successo

22.1 KPI Year 1 (consolidati)

- adozione open source;
- validazione tecnica su scheduler;
- overhead del tracciamento;
- numero di classi workload riconosciute.

22.2 KPI Year 2

- **time-to-profile**: tempo per generare un Micro-OS Profile validato;
- **profile validity**: percentuale di profili che superano benchmark A/B;
- **build success rate**: percentuale di build del profilo che completano con successo;
- **runtime overhead**: overhead del profilo applicato rispetto a kernel generico;
- **custom profile count**: numero di profili verticali prodotti (AI training, inference, HPC, RT, ecc.).

22.3 KPI Year 3+ (microkernel track)

- proof-of-concept su almeno un microkernel candidate;
 - benchmark di isolamento/latenza rispetto a Linux+eBPF;
 - decisione go/no-go sul microkernel come prodotto.
-

23. Prossimi passi per Year 2

1. **Consolidare Year 1**: avere scheduler sched-ext stabili e workload classification affidabile.
 2. **Definire Micro-OS Profile schema**: formato, componenti, metadati, API di build.
 3. **Costruire AI profile assistant**: modulo che suggerisce profili a partire da metriche, con spiegazioni.
 4. **Integrare ottimizzazioni network/memory/I/O**: aggiungere programmi eBPF e configurazioni controllate.
 5. **Pipeline di validazione**: benchmark A/B automatici per ogni profilo generato.
 6. **Documentare casi verticali**: produrre profili dimostrativi per AI training, AI inference, HPC batch, HPC real-time.
 7. **Avviare microkernel research**: studio preliminare su seL4/L4Re/Unikraft, con report interno.
-

Appendice A. Glossario

- **eBPF**: tecnologia per eseguire programmi nel kernel Linux in modo sicuro e verificato.
 - **sched-ext**: framework kernel per implementare classi scheduler come programmi eBPF.
 - **TGID**: Task Group ID, identificativo del processo in Linux.
 - **NUMA**: Non-Uniform Memory Access, architettura memory con nodi distinti.
 - **OpenTelemetry**: standard per osservabilità (metriche, tracing, logging).
 - **cgroup v2**: meccanismo Linux per raggruppare processi e applicare limiti/risorse.
 - **Aya**: framework Rust per sviluppare programmi eBPF.
-

Appendice B. Riferimenti tecnici

- Linux kernel sched-ext documentation
- Aya Rust eBPF framework
- eBPF.io case studies

- OpenTelemetry Metrics API
 - sched-ext repository e scx schedulers di riferimento
-

Documento interno CortexBrain — non committare.

Versione aggiornata con roadmap Year 2 e microkernel research track.